



Empresa Operativa Chile

Crear, operar y controlar una empresa chilena a través del tiempo

Contabilidad y tributación explicables · Reglas versionadas por año · Evidencia obligatoria
Sin servidor · Sin cuentas · Sin telemetría · Los datos no salen del dispositivo

Android · APK

Windows · MSI/EXE

Navegador · PWA

CLI · Node 20+

● EMPRESA REAL

Tu contabilidad, con bitácora de auditoría de cada cambio.

● SANDBOX

Empresa ficticia para practicar. Nunca toca la empresa real.

07 · Persistencia

Documentación de sistema · Empresa Operativa Chile

Documento 07 de la serie

Versión del manifiesto 1.5.0 · commit 25f1a5e

Análisis del 2026-09-22

Documento técnico generado desde docs/system-documentation/. La fuente es el Markdown del repositorio: si este PDF y el Markdown difieren, manda el Markdown. Esta aplicación no presenta ni paga nada ante el SII y no es asesoría tributaria.

07 · Persistencia

La conclusión, primero

No hay base de datos. No hay motor, no hay servidor de datos, no hay ORM, no hay migraciones SQL, no hay cadena de conexión y no hay credencial que proteger. Se buscó: ni un `.sql`, ni un esquema, ni un driver, ni una dependencia de base de datos en ningún `package.json` del repositorio.

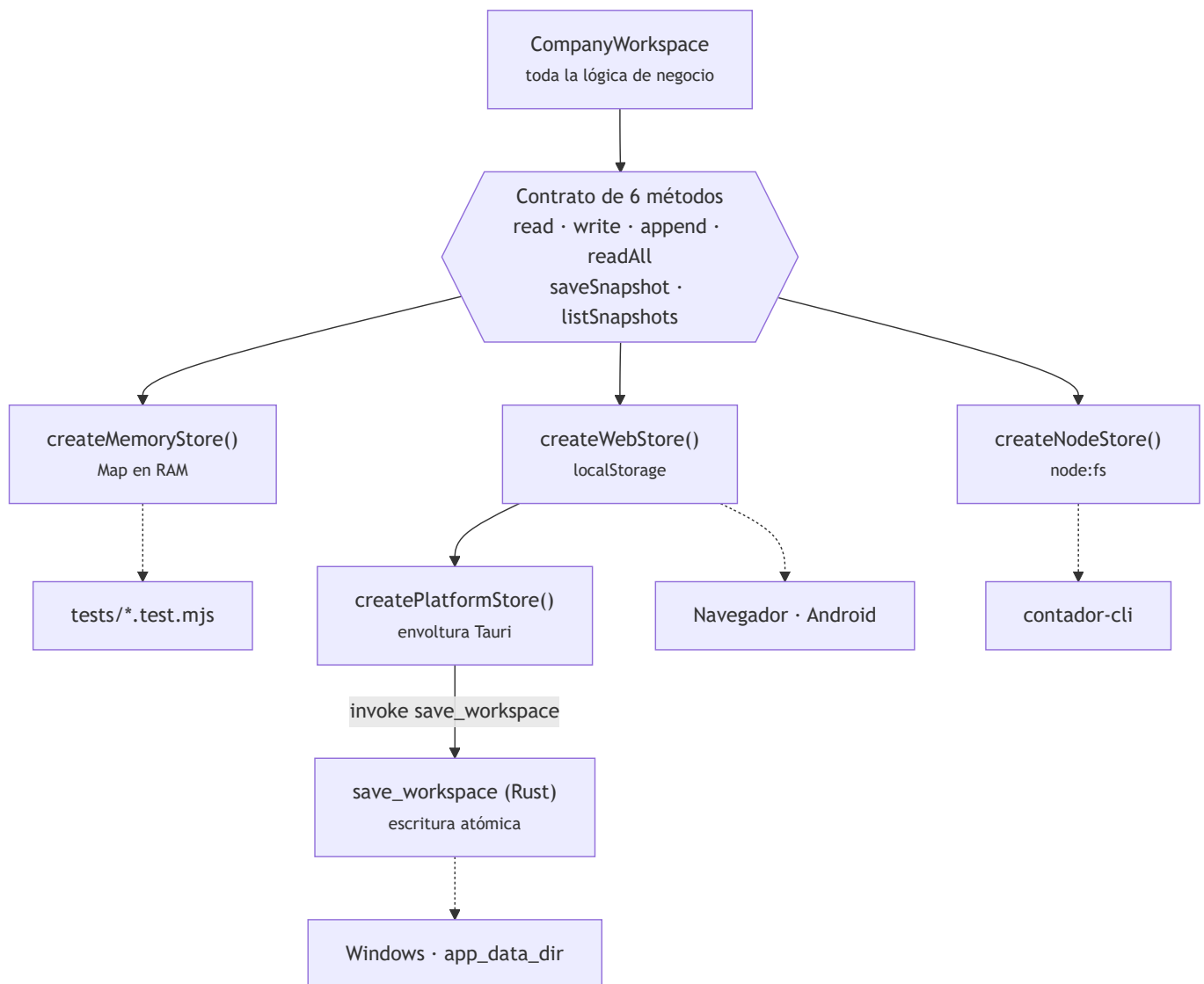
Lo que sí hay es un **contrato de almacén de seis métodos** (`packages/company-operations/store.mjs`) con tres implementaciones. `CompanyWorkspace` —las 1.388 líneas donde vive toda la lógica de negocio— sólo habla ese contrato y no sabe sobre qué está escribiendo.

Este documento cumple lo que se le pide a un capítulo de base de datos: el diccionario de datos está completo, con tipos, claves, relaciones e integridad, aunque la tecnología subyacente sea `localStorage` y archivos JSON en vez de tablas.

Las tres implementaciones del almacén

Implementación	Archivo	kind	Dónde escribe	Quién la usa
Memoria	<code>store.mjs</code> · <code>createMemoryStore()</code>	memory	Tres <code>Map</code> en RAM	Las 13 suites de prueba y las simulaciones desechables
Web	<code>store.mjs</code> · <code>createWebStore()</code>	web	<code>localStorage</code> del navegador	Navegador, WebView de Android, WebView de Windows
Disco	<code>node-store.mjs</code> · <code>createNodeStore()</code>	node	Archivos JSON y NDJSON	La CLI y cualquier proceso Node

Y una cuarta, que es una **envoltura** y no una implementación: `createPlatformStore(mode)` en `apps/web/src/lib/platform.js`. Devuelve el almacén web tal cual en navegador y en Android; bajo Tauri lo envuelve para copiar a disco después de cada escritura.



Qué muestra: que hay un solo punto de bifurcación entre plataformas y está en el almacén, no repartido por la lógica. **Qué no muestra:** que `createNodeStore` y `createPlatformStore` nunca coexisten — son caminos

excluyentes, no capas apiladas.

El contrato, método a método

Método	Firma	Semántica	Qué NO garantiza
<code>read</code>	<code>(key, fallback = null)</code>	Documento JSON completo, o <code>fallback</code>	No distingue «no existe» de «está corrupto»: ambos devuelven <code>fallback</code>
<code>write</code>	<code>(key, value)</code>	Reemplaza el documento entero	Atómico sólo en <code>node</code> y bajo Tauri; en <code>localStorage</code> no hay transacción
<code>append</code>	<code>(key, row)</code>	Agrega una fila a un log	En <code>web</code> lee, empuja y reescribe el arreglo completo — no es un append real
<code>readAll</code>	<code>(key)</code>	Filas del log, en orden de inserción	No pagina ni filtra: devuelve todo
<code>saveSnaps</code> <code>hot</code>	<code>(name, data)</code>	Guarda un respaldo, devuelve su ubicación	El formato de la ubicación cambia por implementación
<code>listSnaps</code> <code>hots</code>	<code>()</code>	Nombres ordenados	No dice cuánto ocupan ni si se pueden leer

La asimetría del `append` importa. En `node-store` un append es una línea nueva en un NDJSON (`fs.appendFileSync`), y el comentario del código explica por qué: reescribir el archivo completo sería la forma más fácil de perder o alterar historial. En `createWebStore` el mismo `append` hace leer, empujar y volver a escribir, es decir ****reescribe todo el arreglo en cada línea de bitácora****. El comportamiento observable es igual; el coste y las propiedades de durabilidad, no. Se recoge como hallazgo en [15 · Riesgos](#).

Dónde acaban los datos, en cada plataforma

Plataforma	Ubicación real	Formato	Sobrevive a...
Navegador / PWA	<code>localStorage</code> del origen	Un <code>string</code> JSON por clave	Cerrar el navegador. No sobrevive a «borrar datos del sitio»
Android (APK)	<code>localStorage</code> de la WebView, dentro del sandbox de la app	Igual	Cerrar la app. No sobrevive a desinstalar ni a «borrar datos»
Windows (Tauri)	<code>localStorage</code> + <code>app_data_dir()/real.json</code> y <code>sandbox.json</code>	Igual + espejo JSON	Reinstalar la app, si no se borra el directorio de datos
CLI / Node	<code><rootDir>/<mode>/*.json</code> + <code>audit.ndjson</code>	Un archivo por entidad	Todo, salvo borrar la carpeta

`app_data_dir()` lo resuelve Tauri a partir del `identifier` de `tauri.conf.json` (`cl.vladimiracuna.empresaoperativa`). La ruta exacta la muestra la propia aplicación en la pestaña

Datos, a través del comando `app_info`. **REQUIERE VALIDACIÓN**: no se ejecutó la app de escritorio en la máquina de análisis, así que la ruta se declara como la que Tauri resuelve, no como una observada.

Cómo se llaman las claves

En `localStorage` cada clave lleva prefijo de aplicación y de modo:

```
empresa-operativa-chile:real:transactions
empresa-operativa-chile:sandbox:transactions
empresa-operativa-chile:real:__snapshots
empresa-operativa-chile:real:snapshot:2026-08-27T12-00-00-000Z
```

El prefijo se arma en `platform.js` y el almacén web antepone `namespace:` a cada clave. **Ese prefijo es toda la separación entre EMPRESA REAL y SANDBOX**: no hay ninguna otra barrera, porque ambos modos comparten origen. Una prueba lo fija (`tests/workspace.test.mjs` , «el almacén web aísla real de sandbox en el mismo origen»).

En disco (`node-store`) cada clave es un archivo, según el mapa `FILES`:

Clave lógica	Archivo	Formato
<code>company</code>	<code>company.json</code>	Objeto
<code>formation</code>	<code>formation.json</code>	Arreglo
<code>transactions</code>	<code>transactions.json</code>	Arreglo
<code>obligations</code>	<code>obligations.json</code>	Arreglo
<code>closed-periods</code>	<code>closed-periods.json</code>	Arreglo de <code>YYYY-MM</code>
<code>period-closes</code>	<code>period-closes.json</code>	Arreglo
<code>audit</code>	<code>audit.ndjson</code>	NDJSON , una línea por hecho
<i>cualquier otra</i>	<code><clave>.json</code>	Objeto o arreglo

Junto al directorio del modo, `createNodeStore` crea siempre `evidence/` y `backups/`.

Hallazgo. `FILES` no incluye `equity-movements`, `annual-closes` ni `municipal-profile`, que son las tres entidades incorporadas en la versión 1.4.0. No se pierden —caen en la rama por defecto y acaban en `equity-movements.json`, `annual-closes.json` y `municipal-profile.json` — pero

`saveSnapshot` sólo copia como archivo suelto lo que está en `FILES`, así que un respaldo hecho con el almacén de disco no incluye esos tres archivos copiados. El `snapshot.json` interior sí lleva los datos, porque sale de `exportAll()`. Ver [15 · Riesgos](#).

Diccionario de datos

Diez entidades. Los nombres de campo se citan **en su forma original** para que buscarlos en el repositorio encuentre exactamente esto. Todos los importes son enteros en pesos chilenos: `money(n) = Math.round(Number(n || 0))`, en `workspace.mjs:27`.

company — ficha de la empresa

Documento único. Lo escribe `saveCompany()`; lo lee casi todo el sistema.

Campo	Tipo	Obligatorio	Notas
<code>legalName</code>	<code>string</code>	No (pero <code>healthCheck</code> lo reclama)	Razón social
<code>rut</code>	<code>string</code>	No (idem)	Validado por <code>packages/company-operations/rut.mjs</code>
<code>commune</code>	<code>string</code>	No	Semilla de la ficha municipal
<code>taxRegime</code>	<code>string</code>	No	Decide si se admite el CPT simplificado
<code>capital</code>	<code>number</code>	No	Campo heredado. Se mantiene sincronizado con <code>capitalProfile.capitalEnterado</code>
<code>capitalProfile</code>	<code>object</code>	No	Ver tabla siguiente
<code>mode</code>	<code>real</code> o <code>sandbox</code>	Lo pone el motor	Se reescribe en cada <code>saveCompany</code>
<code>createdAt</code> / <code>updatedAt</code>	ISO-8601	Los pone el motor	<code>createdAt</code> sólo se fija la primera vez

company.capitalProfile — las magnitudes del capital

No es una entidad aparte: vive **dentro** de `company`. Se normaliza en cada lectura con `normalizeCapitalProfile()` (`capital.mjs:196`), lo que significa que **leer no escribe**: si nadie guarda, el dato original permanece intacto en el almacén.

Campo	Tipo	Valor ausente	Por qué importa
<code>capitalSocial</code>	<code>number</code> o <code>null</code>	<code>null</code>	El del estatuto
<code>capitalSuscrito</code>	<code>number</code> o <code>null</code>	<code>null</code>	Lo que el accionista se comprometió a pagar
<code>capitalEnterado</code>	<code>number</code>	<code>0</code>	Lo efectivamente pagado. Es lo único que es patrimonio
<code>numeroAcciones</code>	<code>number</code> o <code>null</code>	<code>null</code>	
<code>valorNominal</code>	<code>number</code> o <code>null</code>	<code>null</code>	
<code>accionistas</code>	<code>array</code>	<code>[]</code>	<code>name</code> , <code>rut</code> , <code>sharePercent</code> , <code>acciones</code> , <code>capitalSuscrito</code> , <code>capitalEnterado</code>
<code>fechaConstitucion</code>	<code>YYYY-MM-DD</code> o <code>null</code>	<code>null</code>	
<code>fechaInicioActividades</code>	<code>YYYY-MM-DD</code> o <code>null</code>	<code>null</code>	
<code>pendingConfirmation</code>	<code>string[]</code>	<code>[]</code>	Los campos que la app sabe que no sabe
<code>migratedFromLegacyCapital</code>	<code>boolean</code>	<code>false</code>	Marca de la migración del campo <code>capital</code>

`null` no es `0`, y esa distinción es la regla de negocio del documento. `capitalPendingToPay()` devuelve `null` —y no cero— cuando `capitalSuscrito` es desconocido, porque cero significaría «no falta nada», y eso es una afirmación que con ese dato ausente nadie puede hacer.

equity-movements — ledger patrimonial

Arreglo. Lo escriben `addEquityMovement()` y `deleteEquityMovement()`; lo normaliza `normalizeEquityMovement()` (`capital.mjs:310`), que **lanza** si falta la fecha, si el monto no es mayor que cero, o si un aporte en bienes no describe el bien y a su aportante.

Campo	Tipo	Validación	Notas
<code>id</code>	<code>string</code>	—	<code>crypto.randomUUID()</code> , o <code>id-<base36></code> sin WebCrypto
<code>kind</code>	enum	Debe estar en <code>EQUITY_MOVEMENT_KINDS</code>	Cada tipo trae <code>equityEffect</code> y <code>liabilityEffect</code>
<code>date</code>	<code>YYYY-MM-DD</code>	Formato exacto, obligatorio	
<code>amount</code>	<code>number</code>	Mayor que cero	
<code>description</code>	<code>string</code>	—	
<code>contributedBy</code>	<code>string</code> o <code>null</code>	Obligatorio si el tipo exige bien aportado	
<code>assetType</code> / <code>assetDescription</code>	<code>string</code> o <code>null</code>	<code>assetDescription</code> obligatorio en aportes en bienes	
<code>bookValue</code> / <code>taxValue</code>	<code>number</code> o <code>null</code>	—	Valor contable y tributario del bien
<code>evidenceRef</code>	<code>string</code>	—	
<code>status</code>	enum <code>DATA_STATUSES</code>	Cae a <code>DECLARADO</code>	Distingue lo declarado de lo confirmado
<code>origin</code>	enum <code>DATA_ORIGINS</code>	Cae a <code>usuario</code>	
<code>createdAt</code>	ISO-8601	—	

Un aporte y un préstamo del accionista entran por el mismo banco y son opuestos aquí. Por eso son tipos distintos y la aplicación obliga a elegir al registrar, en vez de adivinar meses después.

`transactions` — operaciones de caja

Arreglo. La entidad más escrita del sistema. `addTransaction()` (`workspace.mjs:688`) la valida en tres puertas: tipo soportado, fecha `YYYY-MM-DD` , y **período no cerrado**.

Campo	Tipo	Valor por defecto	Regla de negocio
<code>id</code>	<code>string</code>	UUID	
<code>date</code>	<code>YYYY-MM-DD</code>	Hoy	Su prefijo <code>YYYY-MM</code> es la clave de período
<code>kind</code>	enum de 9 valores	—	Un valor fuera de <code>TRANSACTION_KINDS</code> lanza
<code>description</code>	<code>string</code>	vacío	
<code>net</code> / <code>vat</code> / <code>total</code>	<code>number</code>	<code>total = net + vat</code>	Enteros
<code>documentType</code> / <code>documentNumber</code>	<code>string</code> o <code>null</code>	<code>null</code>	Su ausencia alimenta <code>withoutEvidence</code>
<code>counterpartyRut</code>	<code>string</code> o <code>null</code>	<code>null</code>	Dato personal de un tercero
<code>paid</code>	<code>boolean</code>	<code>false</code>	
<code>deductible</code>	<code>boolean</code>	<code>true</code>	Sólo un <code>false</code> explícito lo desactiva
<code>vatCreditEligible</code>	<code>boolean</code>	<code>true</code>	Si es <code>false</code> , su IVA va a <code>rejectedVat</code>
<code>source</code>	<code>string</code>	<code>manual</code>	
<code>evidence</code>	<code>array</code>	<code>[]</code>	
<code>equityMovementId</code>	<code>string</code> o <code>null</code>	<code>null</code>	Enlace que impide el doble conteo del capital
<code>createdAt</code> / <code>updatedAt</code>	ISO-8601	—	

Los nueve `kind`, con su efecto sobre el IVA:

id	Etiqueta	flow	affectsVat
sale	Venta	in	debit
purchase	Compra del giro	out	credit
expense	Gasto	out	credit
honorarium	Honorario pagado	out	none
capital	Aporte de capital	in	none
shareholder_loan	Préstamo del accionista	in	none
shareholder_loan_repayment	Devolución de préstamo	out	none
owner_withdrawal	Retiro del accionista	out	none
tax_payment	Pago de impuesto	out	none

formation — pasos de constitución

Arreglo, pero **híbrido**: el catálogo de los 9 pasos vive en el código (`FORMATION_STEPS` , congelado con `Object.freeze`) y en el almacén sólo se guarda el estado. `listFormationSteps()` fusiona ambos en cada lectura, dejando ganar lo guardado sobre los valores por defecto.

Consecuencia de diseño, no accidente: cuando se agrega un trámite nuevo al catálogo, las empresas ya creadas lo ven aparecer, en vez de quedarse con una lista congelada del día en que se instalaron. Una prueba lo fija: «el catálogo de pasos se amplía sin perder lo ya registrado».

Campo	Origen	Notas
<code>id</code> , <code>title</code> , <code>authority</code> , <code>evidenceHint</code> , <code>url</code>	Código	Los 9: <code>design</code> , <code>res</code> , <code>rut</code> , <code>start</code> , <code>activities</code> , <code>address</code> , <code>dte</code> , <code>patent</code> , <code>bank</code>
<code>status</code>	Almacén	<code>pending</code> · <code>in_progress</code> · <code>done</code> · <code>blocked</code>
<code>evidenceRef</code>	Almacén	<code>done</code> sin <code>evidenceRef</code> lanza
<code>updatedAt</code>	Almacén	

obligations — obligaciones con vencimiento

Campo	Tipo	Notas
type	string	Obligatorio; <code>upsertObligation</code> lanza sin él
period	string	YYYY-MM o YYYY, según el tipo
dueDate	YYYY-MM-DD	Vencida y sin comprobante ⇒ <code>healthCheck</code> nivel <code>error</code>
status	enum	Marcarla <code>done</code> exige comprobante
evidenceRef	string	

closed-periods y period-closes — el cierre mensual

Dos claves para el mismo hecho, y la separación es deliberada:

- `closed-periods`: arreglo de `YYYY-MM` **ordenado**. Es el índice barato que consulta

`isPeriodClosed()`, y lo consultan `addTransaction`, `updateTransaction` y `deleteTransaction` en cada llamada.

- `period-closes`: arreglo de fotografías completas — `id`, `period`, `at`, `summary`,

`carryForwardIn` y `checklist` —, donde `summary` es el `periodSummary(period)` congelado el día del cierre.

Reabrir borra el período de `closed-periods` **pero no toca** `period-closes`: la fotografía se queda. Y `reopenPeriod(period, reason)` lanza si el motivo viene vacío.

annual-closes — el cierre del ejercicio

Arreglo de objetos **congelados con** `Object.freeze` en el momento de crearse. La entidad más rica del sistema y la que sostiene el argumento del producto.

Campo	Tipo	Por qué está
<code>id</code> , <code>fiscalYear</code> , <code>closingDate</code> , <code>createdAt</code>	—	<code>closingDate</code> siempre <code><año>-12-31</code>
<code>taxRegime</code>	<code>string</code> o <code>null</code>	Copiado de la ficha ese día
<code>assets</code> , <code>liabilities</code> , <code>accountingEquity</code>	<code>number</code>	
<code>balanceOrigin</code>	texto	<code>estimado por la aplicación</code> o <code>declarado por el usuario</code> . Nunca se presenta una estimación como un dato declarado
<code>taxEquity</code> , <code>CPT</code> , <code>CPTMethod</code>	objeto / <code>number</code> / <code>string</code>	Resultado y método usado
<code>taxAdjustments</code>	objeto	
<code>capital</code> , <code>capitalMovements</code>	objeto / arreglo	Movimientos hasta el 31-12
<code>yearSummary</code>	objeto	Agregado del año
<code>municipalPatentBaseFor NextPeriod</code>	objeto	El puente entre un ejercicio y el siguiente
<code>evidence</code> , <code>notes</code>	arreglo / <code>string</code>	
<code>legalRulesVersion</code>	objeto	Ver abajo

`legalRulesVersion` es la decisión de diseño más importante de la persistencia. No se guarda una referencia al archivo de reglas: se guarda una **huella del contenido** —tasas, referencias legales y fechas de verificación— dentro del propio cierre. El comentario del código lo dice sin rodeos: dentro de tres años `rules/2026.json` puede haberse corregido, y el cierre tiene que seguir explicando con qué normas se calculó ese día.

`closeFiscalYear()` **no sobrescribe nunca**: si el ejercicio ya está cerrado, lanza. Rehacerlo exige `reopenFiscalYear(year, reason)` con motivo, y eso queda en la bitácora junto al CPT anterior.

municipal-profile — ficha municipal

Campo	Tipo	Notas
<code>commune</code>	<code>string</code>	Se siembra desde <code>company.commune</code>
<code>rate</code> , <code>status</code>	<code>number</code> / enum	La tasa de la comuna no está en el repositorio : la fija cada municipalidad por ordenanza
<code>initialOwnCapital</code>	<code>number</code> o <code>null</code>	<code>null</code> no es <code>0</code> , otra vez
<code>deductibleInvestments</code>	<code>number</code>	Inversiones en otros negocios con patente
<code>allocatedCapital</code>	<code>number</code> o <code>null</code>	Prorratio entre sucursales
<code>utmPeriod</code>	<code>YYYY-MM</code> o <code>null</code>	La UTM cambia todos los meses
<code>notes</code> , <code>updatedAt</code>		

audit — bitácora append-only

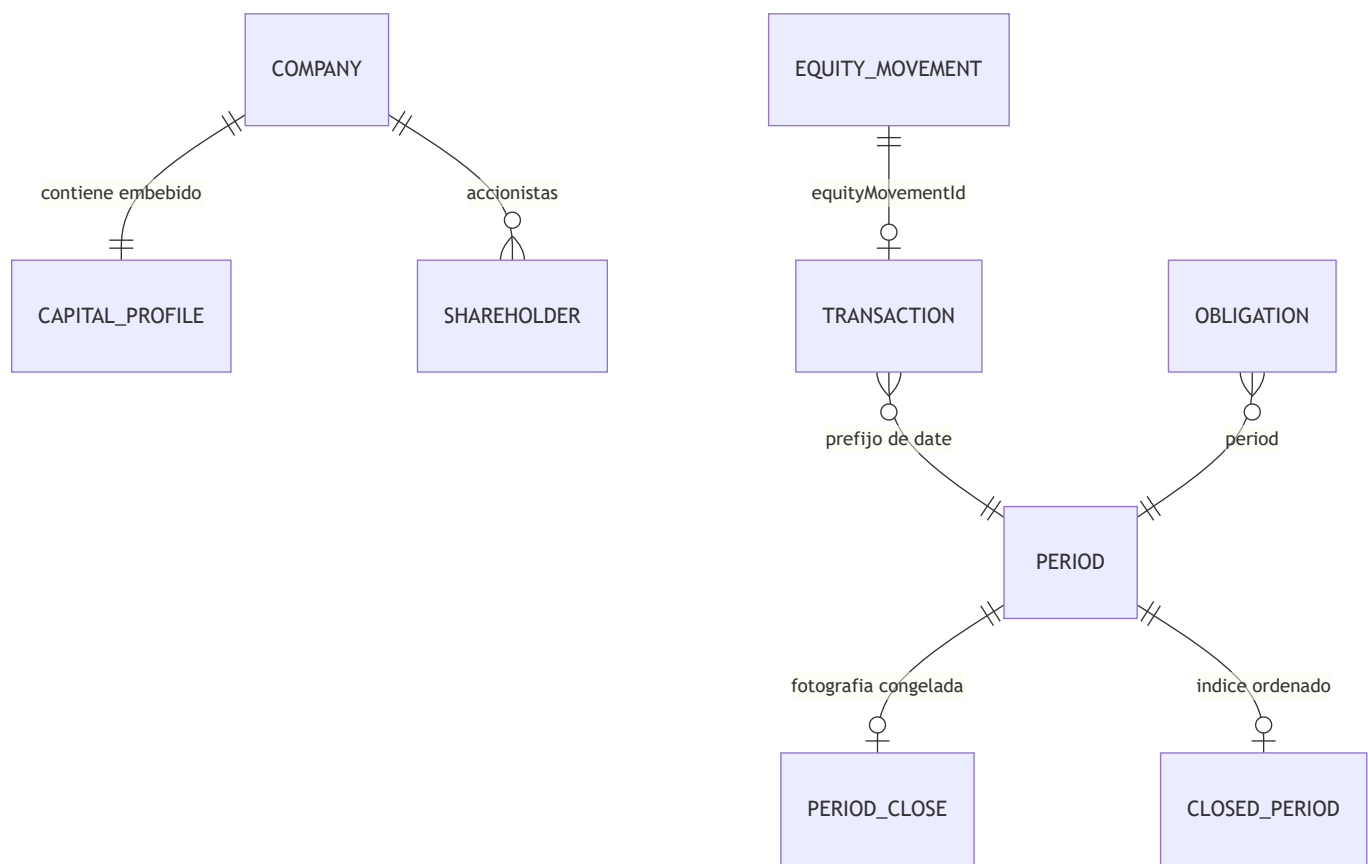
La única entidad que usa `append` y `readAll` en vez de `read` y `write` .

Campo	Tipo	Notas
<code>id</code>	<code>string</code>	UUID
<code>at</code>	ISO-8601	
<code>mode</code>	<code>real</code> o <code>sandbox</code>	
<code>action</code>	<code>string</code>	<code>transaction.added</code> , <code>period.closed</code> , <code>fiscal-year.reopened</code> , ...
<code>detail</code>	objeto	Depende de la acción

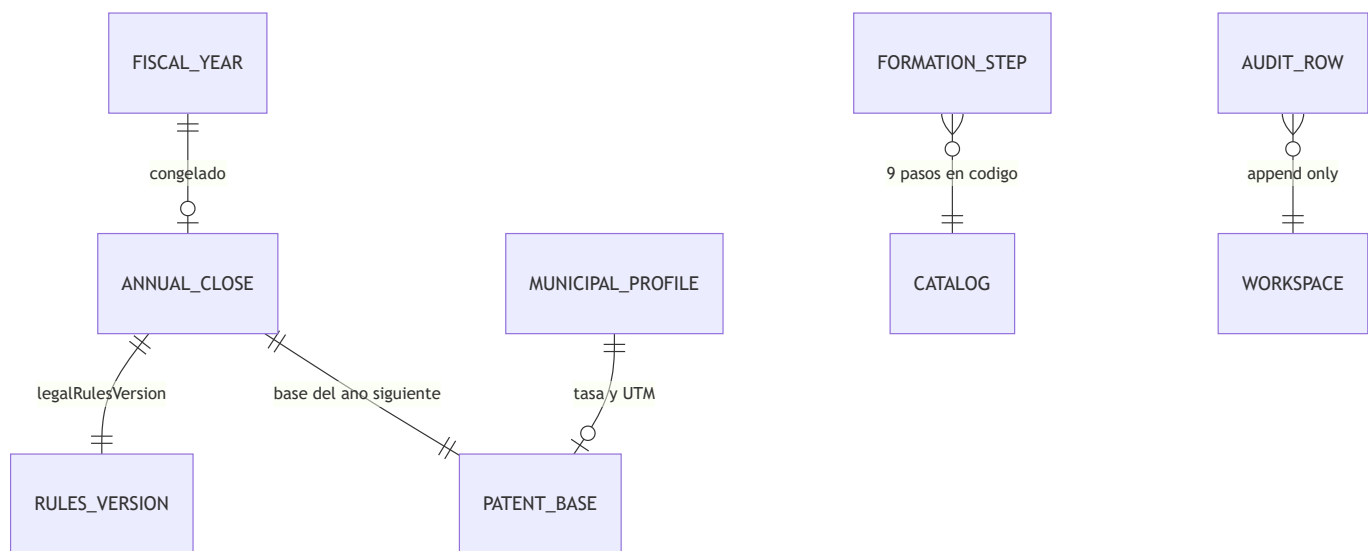
No existe ninguna operación de borrado de la bitácora en toda la clase. No es un olvido: el comentario del código lo declara — si un registro pudiera borrarse sin dejar huella, la bitácora no serviría como evidencia de nada. Una prueba lo fija: «toda mutación queda en la bitácora append-only».

Relaciones e integridad

El día a día — la empresa, su capital y el mes:



El año y la evidencia — lo que se congela y lo que se registra:



Por qué son dos y no uno: en un solo diagrama las trece relaciones salen en una fila de 2.328 píxeles, que impresa en A4 se reduce al 30 % y deja el texto ilegible. El propio generador lo detecta y avisa; partirlo por dominio —el mes por un lado, el ejercicio por otro— es la respuesta a ese aviso.

Qué muestran: que las relaciones existen aunque no haya claves foráneas. **Qué no muestran:** que ninguna de estas relaciones la hace cumplir un motor — todas son código en `workspace.mjs`, y por eso están listadas

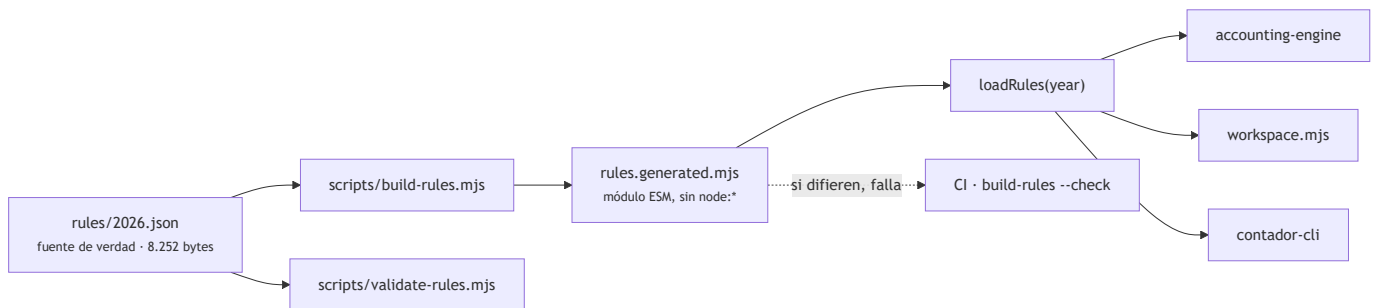
abajo una por una.

#	Regla de integridad	Dónde se hace cumplir	Qué pasa si se viola
1	Un período cerrado no admite alta, edición ni baja	<code>addTransaction</code> , <code>updateTransaction</code> , <code>deleteTransaction</code>	Error
2	Una operación no puede <i>moverse</i> a un período cerrado	<code>updateTransaction</code> , segunda comprobación sobre la fecha ya fusionada	Error
3	Un paso <code>done</code> exige <code>evidenceRef</code> no vacío	<code>updateFormationStep</code>	Error
4	Una obligación <code>done</code> exige comprobante	<code>upsertObligation</code>	Error
5	Reabrir período o ejercicio exige motivo	<code>reopenPeriod</code> , <code>reopenFiscalYear</code>	Error
6	Un ejercicio ya cerrado no se cierra dos veces	<code>closeFiscalYear</code>	Error
7	Un ejercicio cerrado no se pisa al importar en modo fusión	<code>importAll</code>	Se ignora la fila entrante
8	Un aporte enlazado no se cuenta dos veces	<code>equityMovementId</code> + <code>effectiveEquityMovements()</code>	El capital enterado se duplicaría
9	El sandbox no se siembra sobre datos reales	<code>seedSandboxWorkspace</code>	Error
10	El sandbox no se resiembra si ya tiene operaciones	<code>seedSandboxWorkspace</code>	Retorno temprano
11	Un archivo ajeno no se importa	<code>importAll</code> , comprobación de <code>format</code>	Error
12	<code>mode</code> sólo puede ser <code>real</code> o <code>sandbox</code>	Constructor de <code>CompanyWorkspace</code> y <code>safe_mode()</code> en Rust	Error

Lo que ningún mecanismo garantiza: no hay transacciones multi-documento. Cerrar un período escribe `period-closes` y después `closed-periods` en dos operaciones separadas. Si el proceso muere entre ambas —o `localStorage` se llena— queda una fotografía sin su índice. No se observó ocurrir; se declara como límite estructural en [15 · Riesgos](#).

Las reglas tributarias como datos versionados

Es el otro mecanismo de persistencia del sistema, y funciona al revés que el anterior: **de sólo lectura, versionado en Git y compilado durante el build.**



Por qué existe el paso de compilación. Un navegador no puede importar un `.json` de forma portátil, y leerlo con `node:fs` está prohibido por la regla dura de la arquitectura. La salida es un módulo ESM que embebe el objeto, y así el mismo archivo viaja al APK y al ejecutable de Windows.

Por qué un año faltante falla. `loadRules(1999)` lanza `No hay reglas verificadas para el año comercial 1999. Disponibles: 2026`. No degrada al año más cercano. `CONTRIBUTING.md` lo prohíbe por escrito y `tests/rules.test.mjs` lo fija con una prueba. Es la regla de negocio más importante del sistema: un cálculo con la tasa del año equivocado no falla — devuelve un número plausible que aparece meses después como una diferencia con el SII.

Estructura de `rules/<año>.json`

Bloque	Contenido	Campos de procedencia
(raíz)	<code>schemaVersion: 2</code> , <code>country</code> , <code>commercialYear</code> , <code>lastVerified</code> , <code>profile</code> , <code>status</code>	
<code>iva</code>	<code>generalRate: 0.19</code>	<code>source</code> , <code>lastVerified</code> , <code>note</code>
<code>honorarios</code>	<code>retentionRate: 0.1525</code>	idem
<code>ppmProPy me</code>	<code>initialYearRate</code> , <code>upTo50000UfRate</code> , <code>above50000UfRate</code>	idem
<code>idpcProP yme</code>	<code>rate: 0.125</code>	idem
<code>utm</code>	<code>2026-08: 71649</code>	idem
<code>municipalPatent</code>	<code>minUtm</code> , <code>maxUtm</code> , <code>minRate</code> , <code>maxRate</code> , más <code>capitalBasis</code> , <code>deductions</code> , <code>branchAllocation</code> e <code>informationFlow</code>	<code>source</code> , <code>legalReference</code> , <code>effectiveFrom</code> , <code>lastVerified</code> , <code>verificationNote</code>
<code>taxEquity</code>	<code>methods.article41</code> , <code>methods.simplified14D3j</code> , <code>eligibility</code> , <code>initialTaxEquity</code> , <code>warnings</code>	<code>legalReference</code> , <code>source</code> , <code>lastVerified</code> por método
<code>f29</code>	<code>generalDueDay: 12</code> , <code>internetPaidEligibleDueDay: 20</code> , <code>internetNoPaymentDueDay: 28</code>	idem
<code>warnings</code>	6 advertencias de alcance	

Cada tasa lleva su fuente oficial y la fecha en que se verificó. No es decoración: `ruleProvenance(year, ruleName)` la expone a la interfaz, de modo que la aplicación puede decir de dónde salió cada número.

El propio archivo declara su límite más incómodo, y conviene leerlo entero: el `verificationNote` de `municipalPatent` registra que **BCN/LeyChile no respondió el 16-08-2026** y pide reverificar el enlace antes de usar la cifra en una declaración real. Un archivo de reglas que admite lo que no pudo comprobar es exactamente lo que esta documentación pide de sí misma.

Lo que el mecanismo no cubre

- Hay **un solo año**: `availableYears()` devuelve `[2026]`. El sistema no ha vivido todavía un cambio de año, así que la política «un archivo por año» está diseñada y probada, pero **no ejercitada con dos archivos**. **INFERENCIA**: el primer `rules/2027.json` es donde se sabrá si el esquema aguanta.
- `utm` tiene **un solo mes** (`2026-08`). Calcular una patente con la UTM de otro mes exige agregarlo.

- La **tasa de una comuna concreta** no está, y no debe estar: el archivo guarda el rango legal.
-

Datos sensibles

Dato	Dónde vive	Cifrado	Observación
RUT de la empresa	<code>company.rut</code>	No	
RUT de terceros	<code>transactions[].counterpartyRut</code>	No	Dato personal de un tercero
Nombres y RUT de accionistas	<code>capitalProfile.accionistas</code>	No	
Montos, folios y descripciones	<code>transactions</code>	No	Revela relaciones comerciales
Bitácora completa	<code>audit</code>	No	Reconstruye la actividad de la empresa
Respaldos exportados	Archivo JSON elegido por el usuario	No	Legible con cualquier editor

Nada de esto está cifrado en reposo, ni en `localStorage`, ni en el espejo de Windows, ni en los respaldos. `SECURITY.md` lo declara con la misma claridad y lo sitúa en el roadmap. La contrapartida es que tampoco sale del dispositivo: ver [11 · Seguridad](#).

`.gitignore` cubre `.local-data/`, `private-data/`, `real-company-data/`, los `*.sqlite*` y los `empresa-operativa-real-*.json` y `.csv`; el workflow `security.yml` busca en cada push respaldos con la firma `empresa-operativa-chile/backup`, certificados y claves privadas.

Respaldo, restauración y migración

Respaldo. `backup()` nombra el snapshot con la marca de tiempo ISO saneada, guarda `exportAll()` y deja constancia en la bitácora.

Formato portátil. `exportAll()` produce un JSON con `format: empresa-operativa-chile/backup` y `formatVersion: 2`. El mismo archivo se exporta desde Android y se importa en Windows.

Restauración. `importAll(payload, { replace = true })`:

- rechaza cualquier archivo cuyo `format` no sea el propio — prueba: «importar un archivo ajeno se rechaza»;
- acepta `formatVersion 1 y 2`. Los respaldos v1 no traen `equityMovements`, `annualCloses` ni `municipalProfile`, y esos campos quedan vacíos. Romper los respaldos que la gente ya tiene guardados sería peor que limpiar el formato;
- con `replace: false` fusiona por `id` y **respetar los ejercicios ya cerrados**.

Las dos migraciones del sistema, ambas en lectura y sin tocar el almacén:

Migración	Dónde	Comportamiento
Campo <code>capital</code> → <code>capitalProfile</code>	<code>normalizeCapitalProfile()</code>	El valor pasa a <code>capitalEnterado</code> ; <code>capitalSocial</code> y <code>capitalSuscrito</code> quedan en <code>pendingConfirmation</code> . No se inventan
Catálogo de trámites	<code>listFormationSteps()</code>	Los pasos nuevos aparecen sobre el estado guardado

Que las migraciones ocurran **en lectura** tiene una consecuencia que conviene entender: instalar esta versión sobre datos antiguos **no puede corromperlos**, porque hasta que el usuario no guarde, el almacén conserva el dato original.

Lo que no existe, dicho explícitamente

NO IDENTIFICADO — se buscó y no hay evidencia en ningún sentido:

- Motor de base de datos, esquema SQL, migraciones versionadas, procedimientos almacenados, funciones, disparadores, vistas e índices.
 - Pool de conexiones, cadena de conexión, credenciales de base de datos.
 - Caché de datos: **localStorage** es el almacén, no una caché sobre otra cosa. El único caché del sistema es el del service worker, y sólo guarda los archivos de la aplicación.
 - Política de respaldo automática o programada: el respaldo lo dispara el usuario.
 - Retención, purga o archivado de la bitácora. Crece sin techo.
-